

Tecnológico de Monterrey

Proyecto MiBici
Experimentación y Model Registry

Operaciones de aprendizaje automático (MLOps)
TC5061

Equipo 45

Efren Pazzi Manzano - A01326815
Franco Francisco Peña Colorado - A01793289
Leonardo Peraza Alavez - A01840326
Aldo Isai Pérez Hernández - A01797871

Profesor: Iván Reyes Amezcua

25 de octubre de 2026

Repositorio del proyecto: main

1. Alcance y protocolo

Esta segunda entrega continúa la propuesta del equipo: utiliza el mismo repositorio, el experimento **mibici-equipo45**, los viajes de enero-septiembre de 2026 y las mismas preguntas. Se comparan reglas sencillas, modelos candidatos y búsquedas; el mejor modelo de cada pregunta queda recuperable desde el Model Registry.

Pregunta	Tarea y uso	Métrica principal
P1: demanda	Viajes iniciados por estación en la siguiente hora; anticipar demanda operativa.	MAE de validación, menor es mejor.
P2: riesgo virtual	Estado al cierre de la siguiente hora: normal, sin bicicletas o sin anclajes; estudiar priorización del rebalanceo.	F1 macro de validación, mayor es mejor.
P3: patrones	Agrupar estaciones según un perfil horario normalizado de 48 componentes; apoyar planeación.	ARI entre periodos; silhouette y Davies-Bouldin como apoyo.

Entrenamiento: enero-junio. Validación: julio-agosto. Septiembre se conserva para la comparación retrospectiva. Los predictores de P1/P2 usan información disponible antes de predecir. Los perfiles de P3 se construyen por periodo; coordenadas y nombres solo ayudan a interpretar el mapa.

La ingesta obtiene **3,358,273 viajes válidos**. P1 tiene 2,556,542 filas y P2, 2,551,458. P3 tiene 397 estaciones de entrenamiento y 468 de validación. La semilla es 42. Los hashes completos y las cantidades por partición se conservan en los manifiestos.

Cómo se comparan las corridas

P1/P2 exploran 12 combinaciones de boosting por pregunta: learning_rate de 0.05 o 0.1, max_leaf_nodes de 15, 31 o 63 y max_iter de 100 o 300. Cada combinación usa una muestra reproducible de hasta 300,000 filas de entrenamiento y toda la validación. El mejor ajuste se vuelve a entrenar con todas las filas de entrenamiento. P3 compara K-means y Ward para k entre 2 y 8; cada combinación utiliza los perfiles completos.

Las búsquedas solo consultan validación. Los finalistas P1/P2 también registran métricas de septiembre. Ese mes ya se observó en la primera entrega: **no se presenta como una prueba nueva e independiente**. La selección y la promoción automática siguen usando validación.

Código de entrenamiento: mibici/train.py. Protocolo y comandos: guía del MLflow Project.

2. Reproducción y estructura

El archivo raíz **MLproject** declara las entradas ingest, train y verify; **python_env.yaml** fija Python 3.13.15 y reutiliza las dependencias versionadas de requirements.txt. Las funciones de ingesta, variables, entrenamiento, registro y predicción permanecen separadas. Este empaquetado hace explícita la ejecución y su entorno (MLflow, s. f.-b; Treveil et al., 2020).

Servidor y conexión

Con el entorno instalado siguiendo el README, se inicia en una terminal:

```
python -m mlflow server --backend-store-uri sqlite:///mlflow.db
--host 127.0.0.1 --port 5000
```

Los dos renglones anteriores forman un solo comando. En otra terminal se define MLFLOW_TRACKING_URI=http://127.0.0.1:5000 usando la sintaxis del sistema:

Sistema	Comando
Linux / macOS	export MLFLOW_TRACKING_URI=http://127.0.0.1:5000
PowerShell	$env:MLFLOW_TRACKING_URI="http://127.0.0.1:5000"
CMD	set MLFLOW_TRACKING_URI=http://127.0.0.1:5000

Entradas del Project

```
python -m mlflow run . -e ingest --env-manager local
--experiment-name mibici-equipo45
```

```
python -m mlflow run . -e train --env-manager local
--experiment-name mibici-equipo45 -P question=all -P tune=true -P final_test=true
```

```
python -m mlflow run . -e verify --env-manager local
--experiment-name mibici-equipo45
```

Cada bloque es un comando continuo. local reutiliza el entorno activo, instalado con requirements.txt. Con uv instalado se puede usar --env-manager uv para que MLflow cree el entorno declarado. La validación documentada utilizó esa creación automática, un clon limpio y descarga real de los CSV; no se copiaron tablas procesadas.

Los parámetros question, model, learning_rate, max_iter, max_leaf_nodes, l2_regularization, c, n_clusters y seed se pueden pasar con -P. El marcador config toma los valores de config.py sin repetir fechas ni semillas en YAML. Un modelo concreto no modifica champion por defecto; register=true lo solicita explícitamente.

Ejemplo: -P question=p1 -P model=hgb_station -P max_iter=100 -P seed=42. El README incluye comandos equivalentes para los tres sistemas; la ejecución completa se comprobó en Linux, sin afirmar una ejecución de este entorno nuevo en Windows o macOS.

Repositorio: rama main · README · MLproject · entorno declarado.

3. Comparación de P1: demanda

Modelo	lr / hojas / iter.	MAE val.	MAE sept.	RMSE sept.	Mejora val.
Estacional	Regla semanal	0.8988	1.0264	2.1075	0.00%
HGB base	0.1 / 31 / 100	0.7614	0.8719	1.6345	15.28%
HGB + estación	0.1 / 31 / 100	0.7498	0.8620	1.6683	16.58%
HGB ajustado	0.05 / 63 / 300	0.7552	0.8634	1.5564	15.98%

El modelo elegido es **HGB de Poisson con estación categórica**: MAE de validación 0.7498, que reduce el error 16.58% respecto a la regla semanal. Incorporar la estación permite aprender diferencias de nivel entre sitios. Las estaciones menos frecuentes se agrupan en otras, con el agrupamiento aprendido solo en entrenamiento.

En septiembre obtiene MAE 0.8620, RMSE 1.6683, WAPE 59.71% y R^2 0.6425. El MAE mide viajes de error por estación-hora. El WAPE es error absoluto respecto al volumen observado; no equivale a porcentaje de predicciones equivocadas.

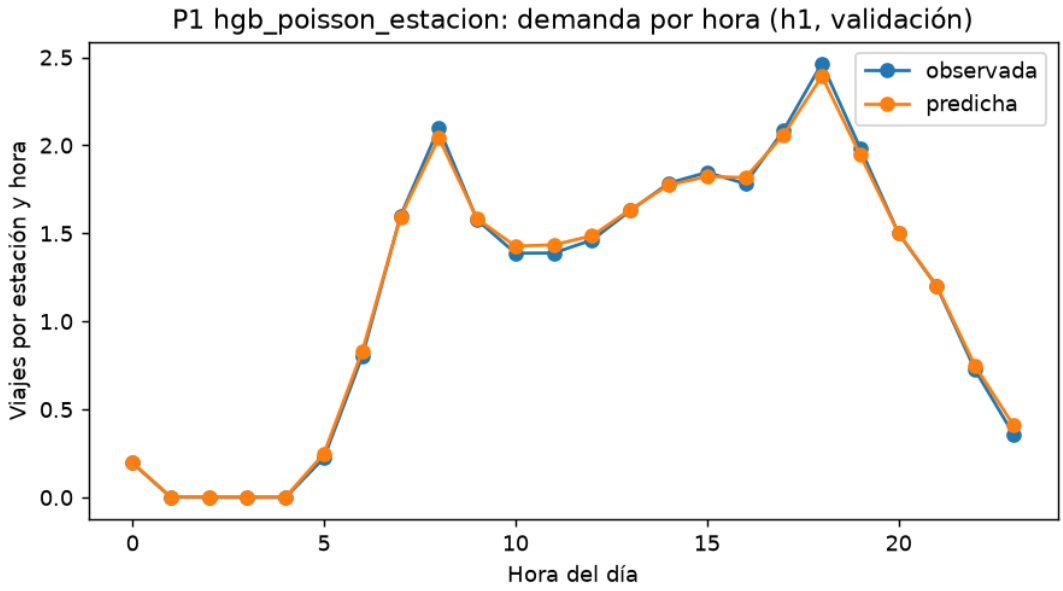


Figura 1. Demanda media observada y predicha por hora en validación. Fuente: artefacto del modelo elegido.

La mejora no elimina errores en estaciones y horas de demanda alta. Una curva promedio cercana puede ocultar esos casos. Antes de apoyar operaciones se deben estudiar errores por estación, horario y volumen. No se atribuye toda la mejora a un único hiperparámetro: cambian las variables y, en el modelo ajustado, varios parámetros a la vez.

La comparación completa de las 12 configuraciones de búsqueda y sus hiperparámetros se conserva en el JSON de resultados y en el padre de MLflow.

4. Comparación de P2: riesgo virtual

Modelo	Parámetros clave	F1 val.	F1 sept.	AP val.	Δ F1 val.
Logística previa	C=1.0	0.8223	0.8114	0.9034	+0.0000
Persistencia	Umbrales 0.1 / 0.9	0.8824	0.8695	0.8130	+0.0600
HGB base	lr=0.1, hojas=31, iter=100	0.8708	0.8574	0.9557	+0.0485
HGB ajustado	lr=0.05, hojas=63, iter=300	0.8723	0.8586	0.9572	+0.0500

La **persistencia** obtiene el mayor F1 macro de validación: 0.8824. Mantiene la clase del inventario virtual actual durante la siguiente hora. Supera a los modelos aprendidos bajo este criterio y cuesta muy poco calcularla; por ello se conserva como champion. La comparación incluye la logística, baseline de la primera entrega, y la nueva regla sencilla.

En septiembre registra F1 macro 0.8695 y balanced accuracy 0.8695. Su AP macro de validación es 0.8130, pero utiliza salidas deterministas 0/1: no se interpretan como probabilidades calibradas. Un modelo con mejor AP puede ser preferible si la decisión futura cambia a ordenar alertas por puntuación.

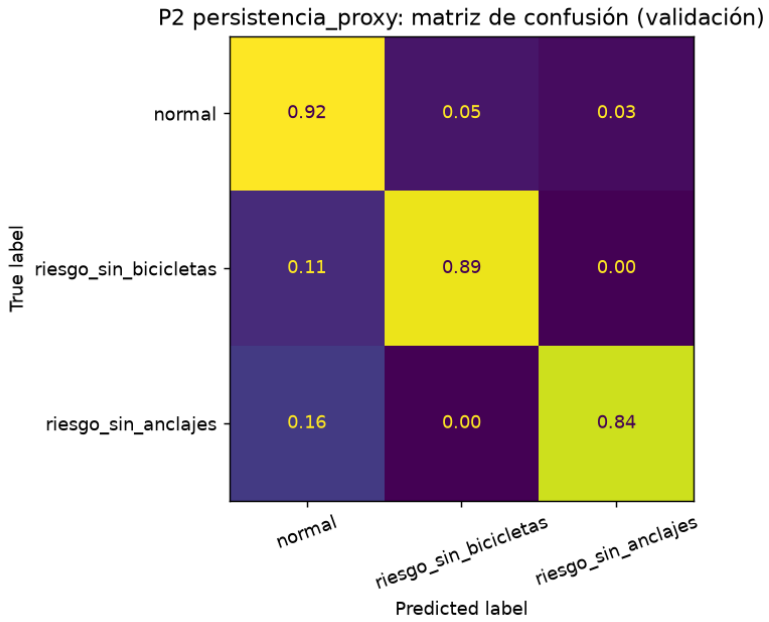


Figura 2. Matriz de confusión de la persistencia en validación. También se guarda una curva precisión-recall por clase.

El resultado mide concordancia con un proxy: la ocupación se simula desde 50%, sin rebalanceos ni inventarios observados. El objetivo y su predictor comparten esa dinámica, lo que favorece la persistencia. No demuestra disponibilidad real ni autoriza automatizar rutas de rebalanceo. La sensibilidad de la ocupación inicial y la validación con inventarios reales siguen siendo límites del caso.

5. Comparación de P3: patrones

Modelo / k	Silhouette train	Davies-Bouldin train	ARI validación
K-means / 2	0.1325	2.4464	0.4578
K-means / 3	0.1407	1.9784	0.5776
K-means / 4	0.1131	2.0854	0.3726
K-means / 5	0.1165	2.1077	0.2923
K-means / 6	0.0963	2.1409	0.3728
K-means / 7	0.0744	2.1223	0.2734
K-means / 8	0.0717	2.0878	0.3384
Ward / 2	0.1040	2.5708	0.1390
Ward / 3	0.1232	2.1388	0.3116
Ward / 4	0.0688	2.4207	0.1760
Ward / 5	0.0755	2.3748	0.1530
Ward / 6	0.0788	2.1728	0.1518
Ward / 7	0.0838	2.0835	0.1526
Ward / 8	0.0671	2.1691	0.1831

El baseline de la propuesta es K-means con k=3; el barrido incluye ese mismo ajuste y 13 alternativas. Se elige **K-means, tres grupos**, por su mayor ARI entre periodos. Un silhouette superior para otro k no basta si disminuye la estabilidad temporal. Se revisan también el tamaño de los grupos y su perfil horario.

El elegido obtiene silhouette 0.1407, Davies-Bouldin 1.9784 y ARI 0.5776. Los grupos contienen 213, 33 y 151 estaciones. La separación es baja y la estabilidad parcial: no se afirma que existan tres tipos naturales o definitivos de estación.

El modelo congelado asigna las 468 estaciones de validación con silhouette 0.1228 y Davies-Bouldin 2.2115. Estas métricas se llaman val_assignment_* y no se confunden con el ARI histórico, que compara ajustes independientes sobre las estaciones comunes. No se calcula una etiqueta real ni exactitud supervisada.

K-means predice por centroide de entrenamiento. Ward no ofrece predict: su adaptador usa el grupo del perfil de entrenamiento más cercano mediante 1-NN. La regla queda documentada y no vuelve a ajustar Ward al recibir una estación nueva.

6. Selección, costos y ciclo de vida

Pregunta / elegido	Ajuste (s)	Inferencia / 1,000 filas (s)	Criterio
P1 / hgb_poisson_estacion_h1	2.8623	0.003370	Menor MAE de validación
P2 / persistencia_proxy_h1	0.0179	0.000187	Mayor F1 macro de validación
P3 / kmeans_perfil48	0.0114	0.000827	Mayor ARI entre periodos

Los tiempos corresponden a esta máquina y ejecución; el tiempo de ajuste excluye la exportación de artefactos. No son un benchmark entre equipos. La regla de persistencia simplifica P2. P1 requiere más ajuste y procesamiento de estación para reducir MAE. En P3, k=3 conserva una interpretación manejable y la mejor estabilidad registrada. No se utilizan tiempos para ocultar diferencias de calidad.

En P1/P2 hay una sola partición temporal: no se afirma estabilidad estadística por repetir muchas semillas. P3 aporta el ARI entre periodos. Conviene añadir ventanas temporales múltiples y revisar errores por estación antes de uso real. La separación entre desarrollo, validación y operación ayuda a controlar riesgos y cambios del sistema (Lakshmanan et al., 2020; Treveil et al., 2020).

Un modelo registrado por pregunta

Pregunta	Nombre del Registry	Versión / alias
P1	mibici-p1-demanda	1 / champion
P2	mibici-p2-riesgo	1 / champion
P3	mibici-p3-grupos	1 / champion

Se usa el alias **champion** porque identifica la versión vigente sin fijar su número en el consumidor. Antes de moverlo, el código crea un alias temporal, carga el modelo y reproduce predicciones guardadas. Si la carga falla, conserva el champion anterior. El alias representa una selección local para el ejercicio; no se desplegó un servicio de inferencia (MLflow, s. f.-d).

Cada versión registra descripción, experimento, corrida de origen, commit, data_version y feature_version. Se conserva el nombre inicial de P1/P2 por compatibilidad; la etiqueta equipo=45 identifica a los tres modelos. Se carga, por ejemplo, models:/mibici-p1-demanda@champion.

Los CSV, Parquet, mlflow.db, almacenes de artefactos y modelos serializados se excluyen de Git. El registro de entrada contiene metadatos y digest de variables agregadas por estación, sin registros de personas usuarias. Los manifiestos documentan la versión sin redistribuir los viajes (MLflow, s. f.-a).

7. Evidencia: búsquedas y comparación

Las capturas corresponden al servidor real de validación. Cada búsqueda tiene un padre y una corrida hija por combinación; el Project añade el padre de ejecución. Los hijos guardan su tamaño de muestra, parámetros, métricas y artefactos.

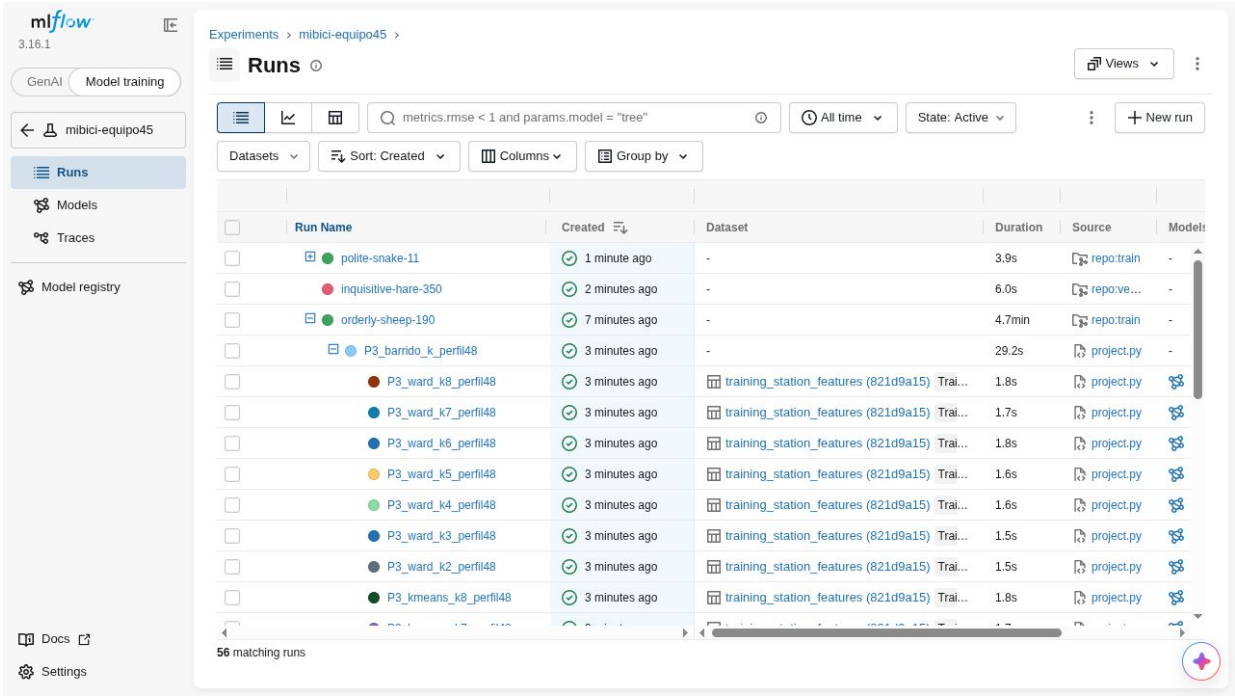


Figura 3. Ejecución del Project y estructura de corridas anidadas.

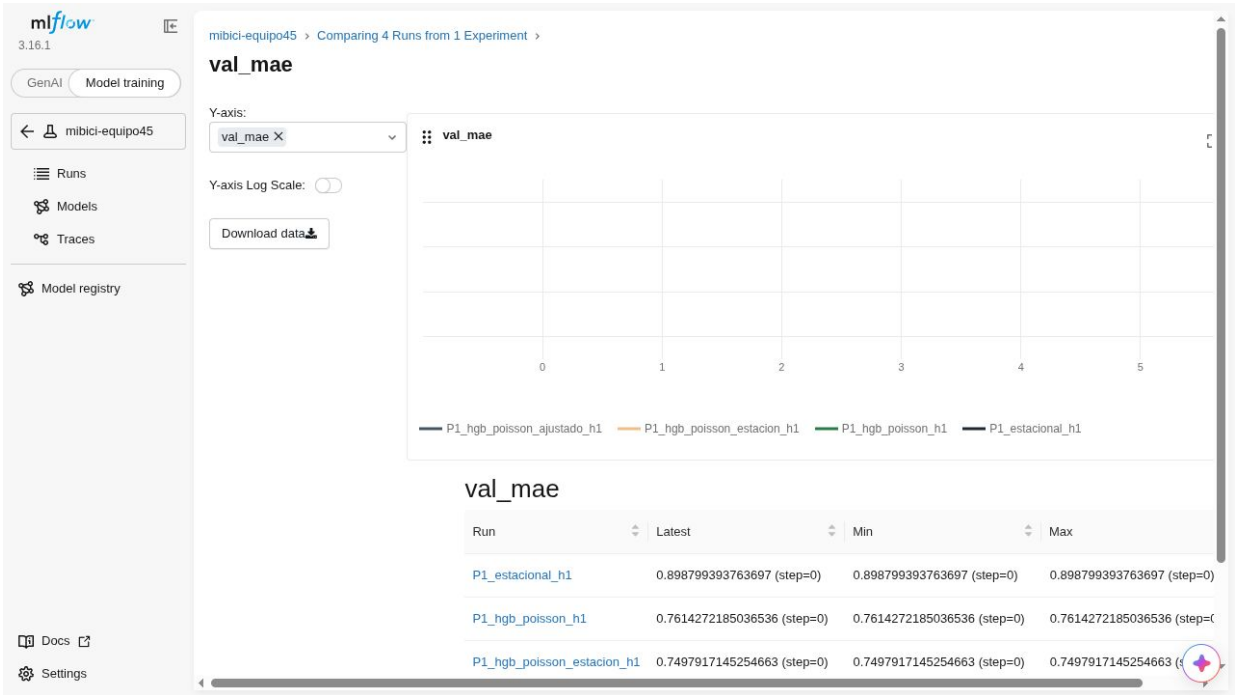


Figura 4. Comparación de candidatos de P1 desde la interfaz de MLflow. Los resultados precisos se incluyen en el JSON de la entrega.

8. Evidencia: artefactos y modelo

Las candidatas y los baselines guardan modelo con firma y ejemplo de entrada; manifiestos en data/; configuración, dependencias, MLproject y parámetros efectivos en config/; figuras en figuras/ y tablas cuando corresponde. En MLflow 3 el modelo también se consulta en Logged Models; no se interpreta su ausencia como subcarpeta del run como falta de registro (MLflow, s. f.-c).

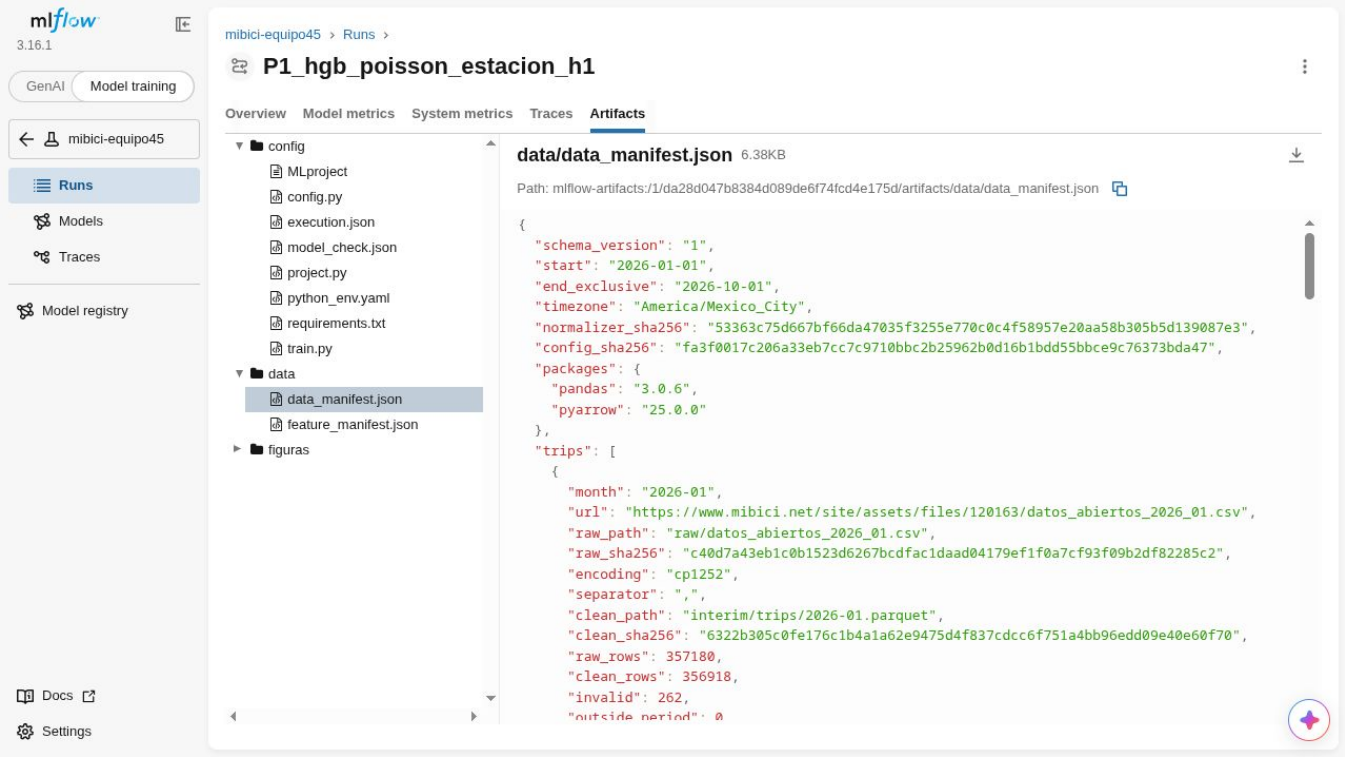


Figura 5. Artefactos de una candidata y manifiesto registrado.

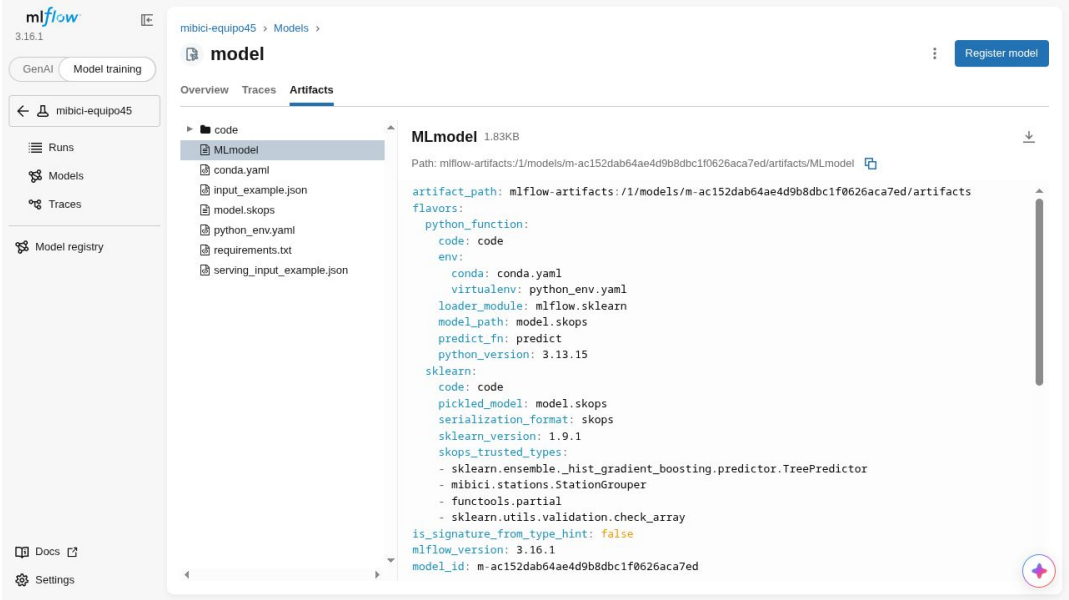


Figura 6. Modelo guardado con firma y ejemplo de entrada.

9. Evidencia: Registry y comprobación

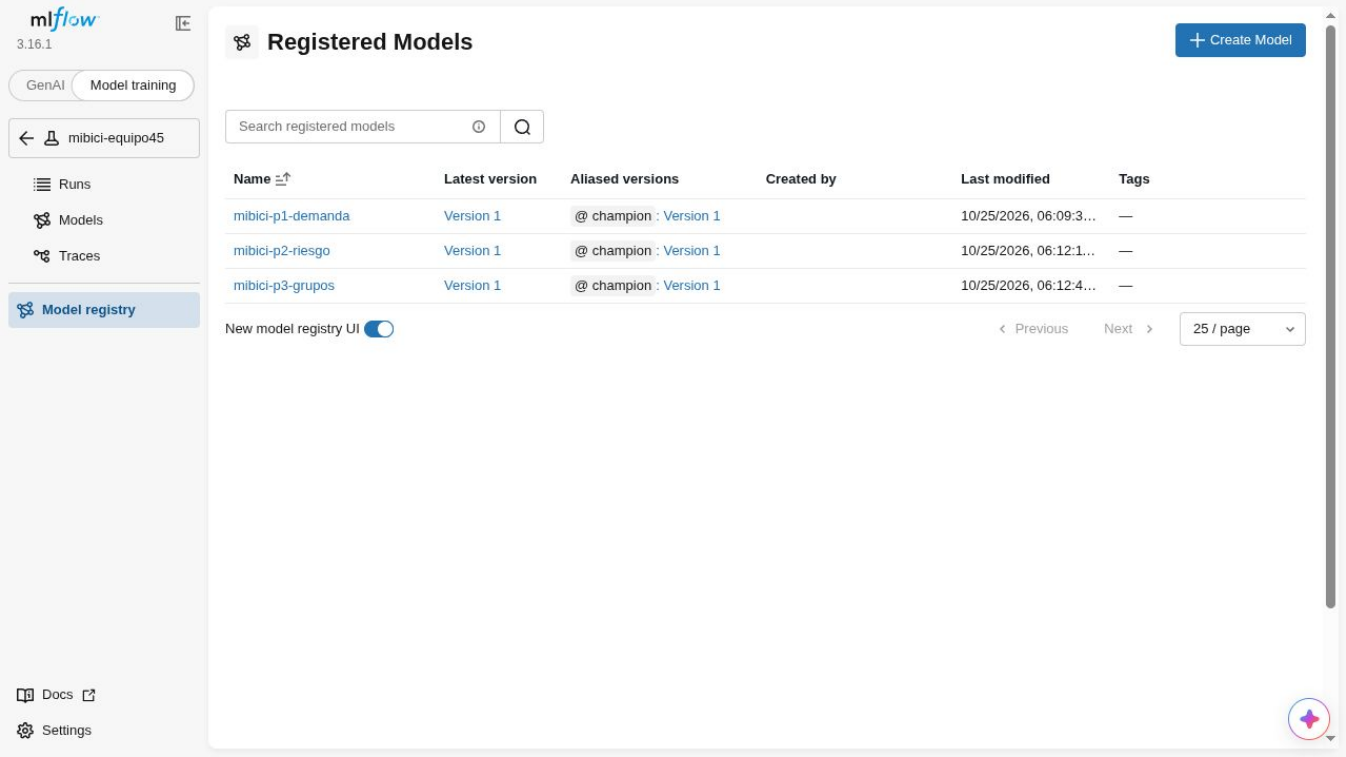


Figura 7. Los tres modelos registrados con alias champion.

Comprobaciones ejecutadas

El clon limpio descargó y reconstruyó los datos con el entorno declarado. Los 74 tests de unidad e integración pasaron. La ejecución completa registró 52 corridas de entrenamiento y búsqueda, incluido el padre del Project, y 48 modelos con firma, ejemplo, configuración y manifiestos. La ingesta y la comprobación del Registry tienen sus propias corridas del Project.

Los tres modelos se cargaron por nombre y alias. Se reprodujeron los ejemplos guardados y las métricas de validación del modelo recuperado; los conteos evaluados fueron 665,747 filas de P1, 662,771 de P2 y 468 estaciones de P3. Los comandos terminaron sin errores.

Commit que produjo los resultados: 92e305f3de71. Las capturas y el reporte se agregan después de esa ejecución; no alteran las predicciones. Los enlaces de lectura del documento apuntan a main y la huella del experimento identifica el código exacto.

Las versiones cambiaron al centralizar los valores de entrenamiento en config.py. La reconstrucción conserva los hashes de CSV y Parquet: no cambia el contenido ni las particiones. Las corridas anteriores mantienen sus versiones originales.

10. Linaje del modelo P1

mibici-p1-demanda · versión 1 · alias champion

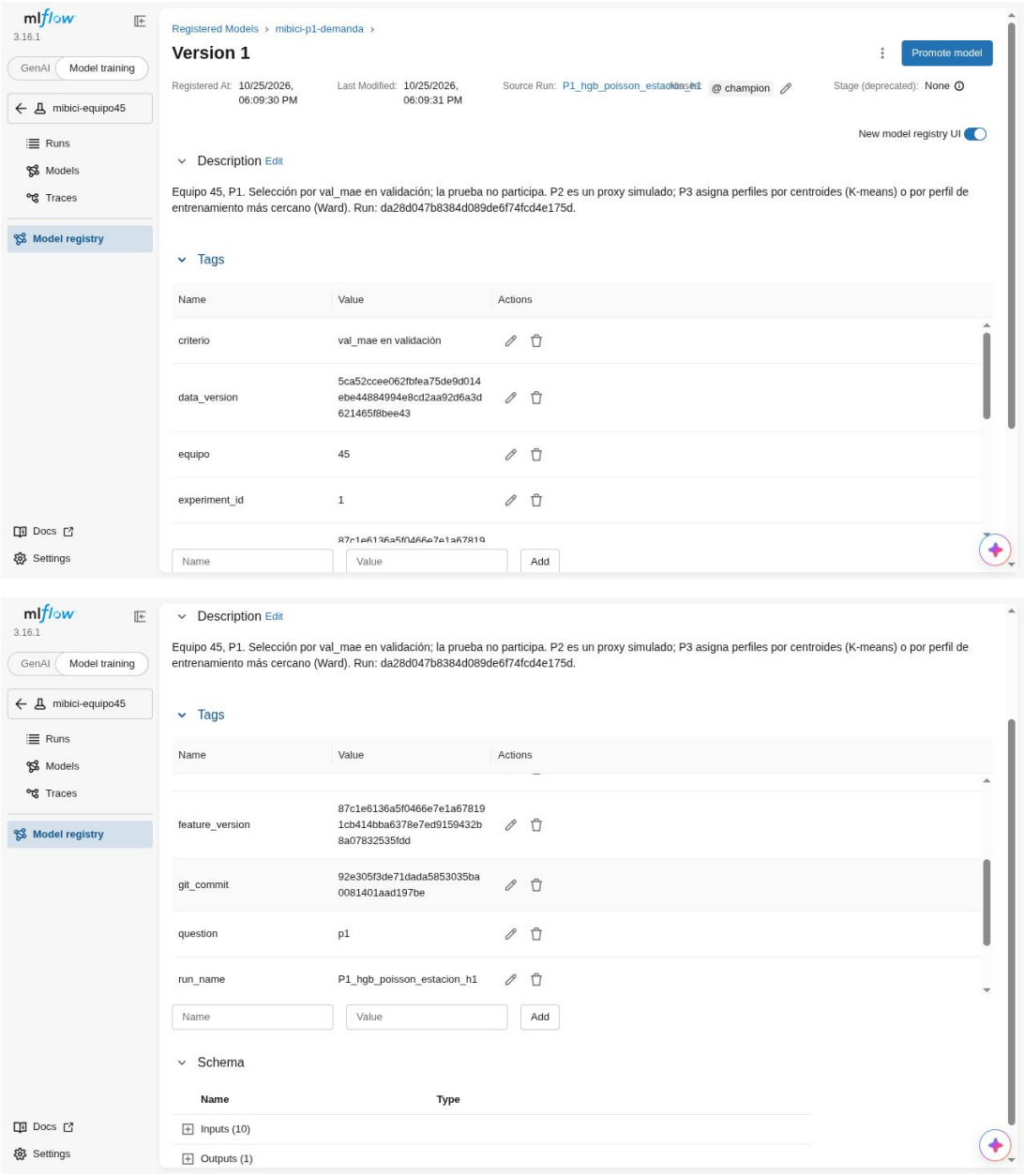


Figura 8. Descripción y etiquetas de la versión seleccionada en la interfaz del Model Registry.

Run: da28d047b8384d089de6f74fcd4e175d
Commit: 92e305f3de71dada5853035ba0081401aad197be
Datos: 5ca52ccee062fbfea75de9d014ebe44884994e8cd2aa92d6a3d621465f8bee43
Variables: 87c1e6136a5f0466e7e1a678191cb414bba6378e7ed9159432b8a07832535fdd

10. Linaje del modelo P2

mibici-p2-riesgo · versión 1 · alias champion

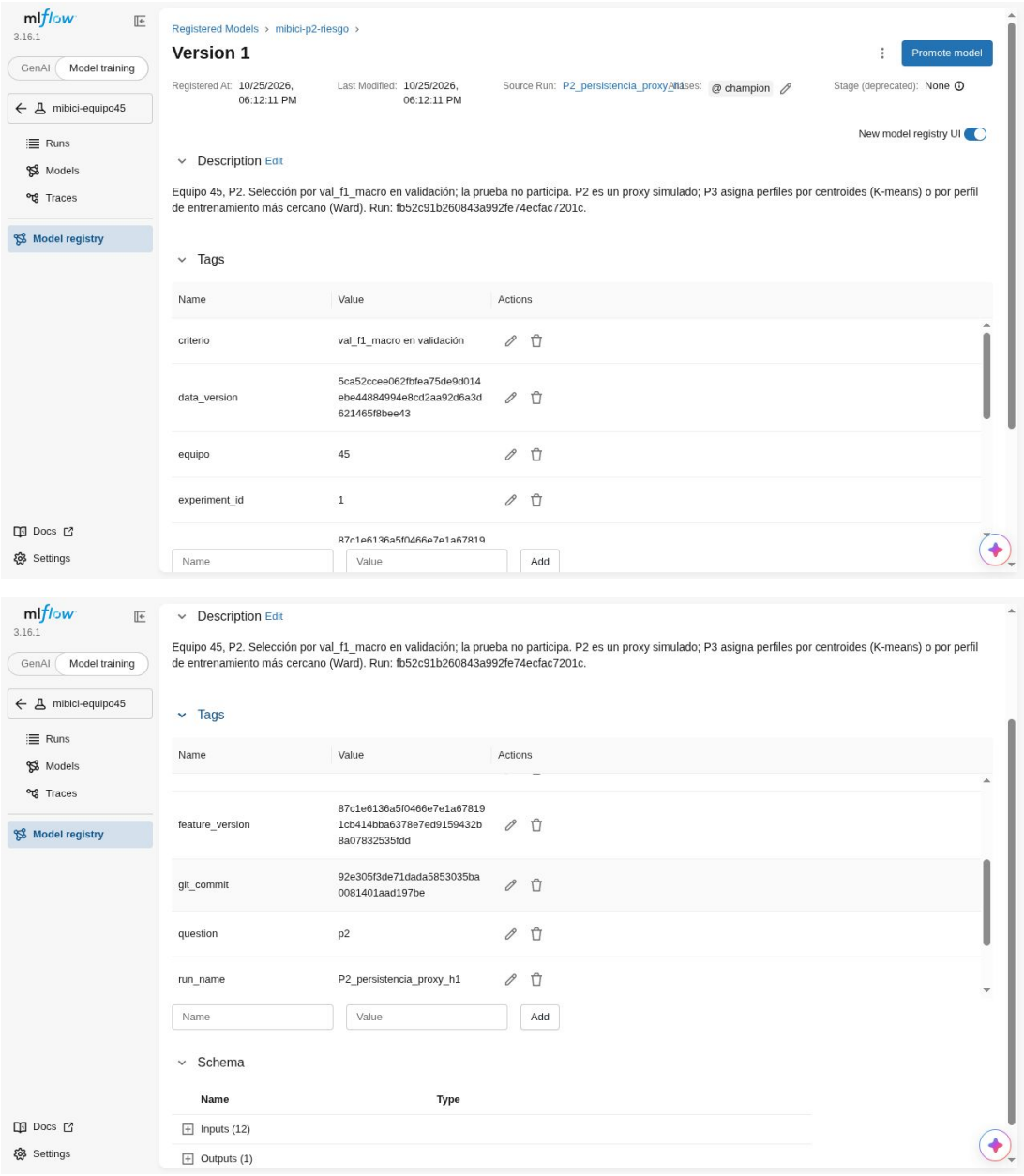


Figura 9. Descripción y etiquetas de la versión seleccionada en la interfaz del Model Registry.

Run: fb52c91b260843a992fe74ecfac7201c
Commit: 92e305f3de71dada5853035ba0081401aad197be
Datos: 5ca52ccee062fbfea75de9d014ebe44884994e8cd2aa92d6a3d621465f8bee43
Variables: 87c1e6136a5f0466e7e1a678191cb414bba6378e7ed9159432b8a07832535fdd

10. Linaje del modelo P3

mibici-p3-grupos · versión 1 · alias champion

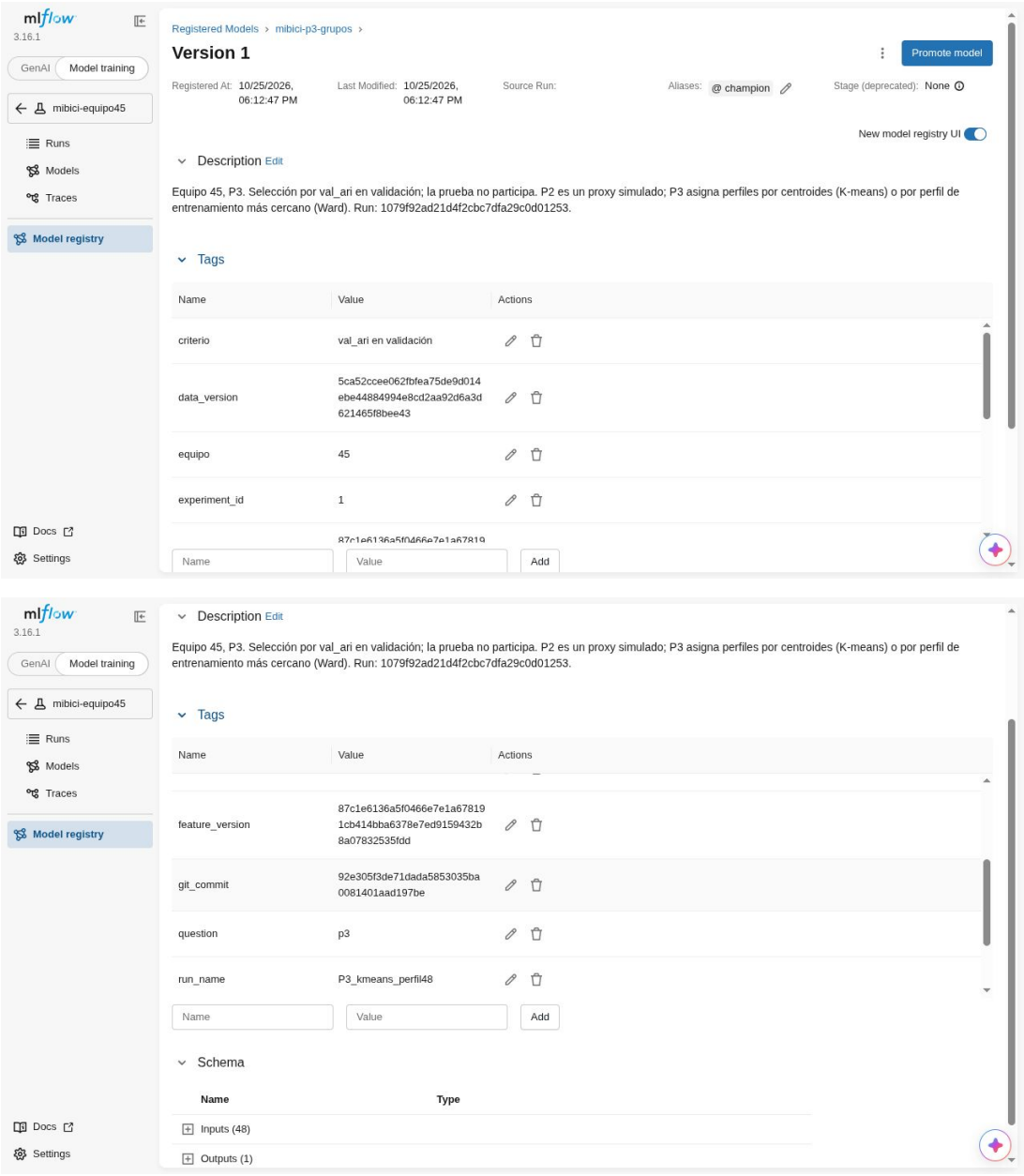


Figura 10. Descripción y etiquetas de la versión seleccionada en la interfaz del Model Registry.

Run: 1079f92ad21d4f2cbc7dfa29c0d01253
Commit: 92e305f3de71dada5853035ba0081401aad197be
Datos: 5ca52ccee062fbfea75de9d014ebe44884994e8cd2aa92d6a3d621465f8bee43
Variables: 87c1e6136a5f0466e7e1a678191cb414bba6378e7ed9159432b8a07832535fdd

11. Conclusiones

La experimentación muestra que el modelo más complejo no gana en todas las preguntas. P1 mejora al representar la estación; P2 conserva una regla simple con mayor F1 macro y menor costo; P3 mantiene tres grupos por estabilidad entre periodos. Se comparan métricas acordes con la decisión y se documentan límites del proxy, de la cobertura temporal y de la separación entre perfiles.

El aporte de MLOps es hacer comprobables esas decisiones: un Project ejecutable, dependencias declaradas, datos con hashes, búsquedas anidadas, artefactos organizados y versiones recuperables por alias. El linaje permite vincular cada resultado con su código y datos. Esto prepara el consumo posterior de los modelos, sin confundir un alias local con un sistema operativo ya desplegado.

Referencias

Lakshmanan, V., Robinson, S., & Munn, M. (2020). *Machine learning design patterns: Solutions to common challenges in data preparation, model building, and MLOps.* O'Reilly Media.

MiBici. (s. f.). *Datos abiertos.* https://www.mibici.net/es/datos-abiertos/

MLflow. (s. f.-a). *MLflow Dataset Tracking.* https://mlflow.org/docs/latest/ml/dataset/

MLflow. (s. f.-b). *MLflow Projects.* https://mlflow.org/docs/latest/ml/projects/

MLflow. (s. f.-c). *MLflow Tracking.* https://mlflow.org/docs/latest/ml/tracking/

MLflow. (s. f.-d). *Model Registry workflows.* https://mlflow.org/docs/latest/ml/model-registry/workflow/

Treveil, M., Omont, N., Stenac, C., Lefevre, K., Phan, D., Zentici, J., Lavoillotte, A., Miyazaki, M., & Heidmann, L. (2020). *Introducing MLOps: How to scale machine learning in the enterprise.* O'Reilly Media.